Digital Systems

Lecture 4

Assembly Programming

Mark van der Wilk

Hilary Term 2026

University of Oxford
Department of Computer Science

Following the course of past years by Mike Spivey

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
-
-
-
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
-
-
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
- Disassembly: From machine code back to assembly.
-
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
- Disassembly: From machine code back to assembly.
- Linking: Giving subroutines absolute addresses.
-

Last Lecture: Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
- Disassembly: From machine code back to assembly.
- Linking: Giving subroutines absolute addresses.
- Startup procedure

Forgot to mention...

What is the format that we actually *send* the final code in?

-
-
-

Forgot to mention...

What is the format that we actually *send* the final code in?

- `more func.hex`

  I.e. *text*-encoded hexadecimal. The "programmer chip" on the micro:bit turns this into binary, before sending it to the Nordic ARM chip.
-
-

Forgot to mention…

What is the format that we actually *send* the final code in?

- `more func.hex`

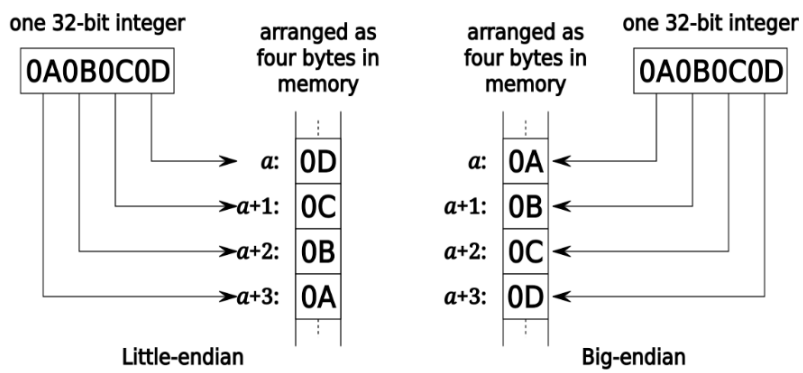
  I.e. *text*-encoded hexadecimal. The "programmer chip" on the micro:bit turns this into binary, before sending it to the Nordic ARM chip.
- Can we still see `18404770` anywhere?
-

Forgot to mention...

What is the format that we actually *send* the final code in?

- `more func.hex`

  I.e. *text*-encoded hexadecimal. The "programmer chip" on the micro:bit turns this into binary, before sending it to the Nordic ARM chip.
- Can we still see `18404770` anywhere?
- No! Instead, we see `40187047`. Stored "little-endian".



If you want to practice + learn more about this:

- Additional questions on this in `problem-sheets/prob1-new.pdf`.
- Answers are included, so good study material.

? Why is there a separation between the compiler and linker?

? Why is there a separation between the compiler and linker?

So you can pre-compile parts of your code.

- Compiling takes time! Some projects take hours to compile.
  Only compiling parts that have changed, saves time.

? Why is there a separation between the compiler and linker?

So you can pre-compile parts of your code.

- Compiling takes time! Some projects take hours to compile.
  Only compiling parts that have changed, saves time.

Simplification and encapsulation of tasks:

- Compiler only needs to know what CPU *core* we're running on, i.e. which instructions our CPU (Cortex-M0) has.
- Linker needs to know details about specific chip, e.g. the memory map.

Digital Systems

Lecture 4

Assembly Programming

Mark van der Wilk

Hilary Term 2026

University of Oxford
Department of Computer Science

Following the course of past years by Mike Spivey

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
muls r0, r3, r0
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
muls r0, r3, r0
bx lr
```

Demo: Coding Assembly

```
@ (a^2 - b^2) = (a+b) * (a-b)
@ at calling a is stored in r0
@            b is stored in r1
adds r3, r0, r1     @ (a + b) stored in r3
subs r0, r0, r1     @ (a - b) stored in r0
muls r0, r3, r0
bx lr
```

What Instructions Can I Use?

How to look up allowed instructions?

We have three summaries:

- "Rainbow Chart" (included in exam)
- List of common instructions (included in exam)
- Arm Thumb Quick Reference Guide (official from ARM)

Definitive document is ARM v6-M Architecture Reference Manual.

Let's look at details of `add` instruction.

- Index: A.6 Thumb Instruction Details
- A.6.7 Alphabetical list of ARMv6-M Thumb instructions
- Various flavours of `add` instructions.

Different Flavours of Addition

Operation		§	Assembler	Updates	Action	Notes
Add	Immediate 3		`ADDS Rd, Rn, #<imm>`	N Z C V	Rd := Rn + imm	imm range 0-7.
	All registers Lo		`ADDS Rd, Rn, Rm`	N Z C V	Rd := Rn + Rm	
	Hi to Lo, Lo to Hi, Hi to Hi		`ADD Rd, Rd, Rm`		Rd := Rd + Rm	Not Lo to Lo.
	Any to Any	T2	`ADD Rd, Rd, Rm`		Rd := Rd + Rm	Any register to any register.
	Immediate 8		`ADDS Rd, Rd, #<imm>`	N Z C V	Rd := Rd + imm	imm range 0-255.

Optimise The Common Case

- Design principle: Make the common case fast, while leaving uncommon case possible.
- Amdahl's law: The maximum achievable speed-up of an optimisation is limited by how often the optimised part is used.

For the exam:

Familiarise yourself with the instructions, but no need to memorise.

More Assembly Programming

Naive Multiplication in C

```
unsigned func(unsigned a, unsigned b) {
    unsigned x = a, y = b, z = 0;

    /* Invariant: a × b = x × y + z */
    while (y != 0) {
        y = y - 1;
        z = z + x;
    }

    return z;
}
```

Naive Asm Translation of Naive Multiplication in C

```
func:                       @ x in r0, y in r1
    movs r2, #0             @ z = 0
loop:
    cmp r1, #0              @ if y == 0
    beq done                @     jump to done
    subs r1, r1, #1         @ y = y - 1
    adds r2, r2, r0         @ z = z + x
    b loop                  @ jump to loop
done:
    movs r0, r2             @ return z
    bx lr
```

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
-
-
-
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
-
-
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
-
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
-
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
- `layout regs`
-
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
- `layout regs`
- `break *func`
-

Demo: Programming, Running, Debugging

How do we check whether the programming is running as expected?

Directory `/lab1`

- `make mul1.hex`
- Run code. `minicom` for communication.
- `./debug mul1.elf`
- `layout asm`
- `layout regs`
- `break *func`
- `stepi`

Recap

ⓘ Understanding Compilation Process

What is specified after compilation, and what after linking.

💪 Programming in Assembly

- Assembly has quirks: They are there for a reason! (Remember?)
- Looking them up in references.

💪 Using a Debugger

- Stepping through assembly, instruction-by-instruction.
- Viewing effect on registers.